



Linux
Professional
Institute

DEVOPS
TOOLS ENGINEER

TOPIC-NINE

LEARNING ANSIBLE



Arash Foroughi

[linkedin.com/in/arashforoughi/](https://www.linkedin.com/in/arashforoughi/)



Ansible Fundamentals



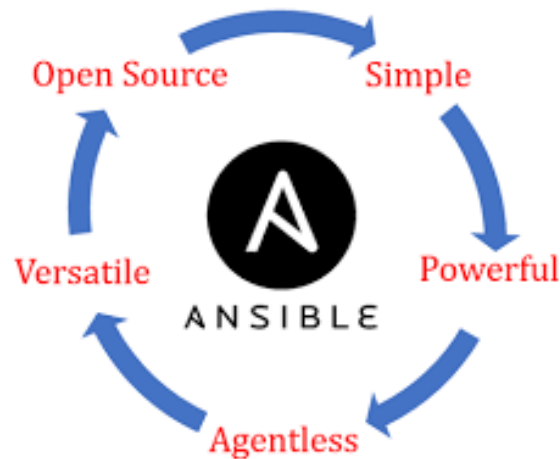
What is Configuration Management

- A process for **maintaining computer systems**, servers & softwares.
- A systems engineering process for **establishing consistency** of a product's attributes.
- Configuration management can be used to **maintain OS configuration files**.
- **Ansible, Bcfg2, CFEngine, Chef, Nix, Otter, Puppet, Quattor, SaltStack, Terraform, Pulumi & Vagrant.**
- Many of these systems utilize **Infrastructure as Code (IaC)** to maintain configurations.



Configuration Management Benefits

- **Rolling out new configs** to all applicable servers.
- **Centralizing** modify all server configurations.
- Manage servers easier in different **groups & subgroups**.
- Define automate configurations like **patches & updates**.
- Identify **outdated & old configs** & define to update them.
- Setups are free of **human-mistake errors**.
- Makes **infrastructure more flexible** and ready to use.



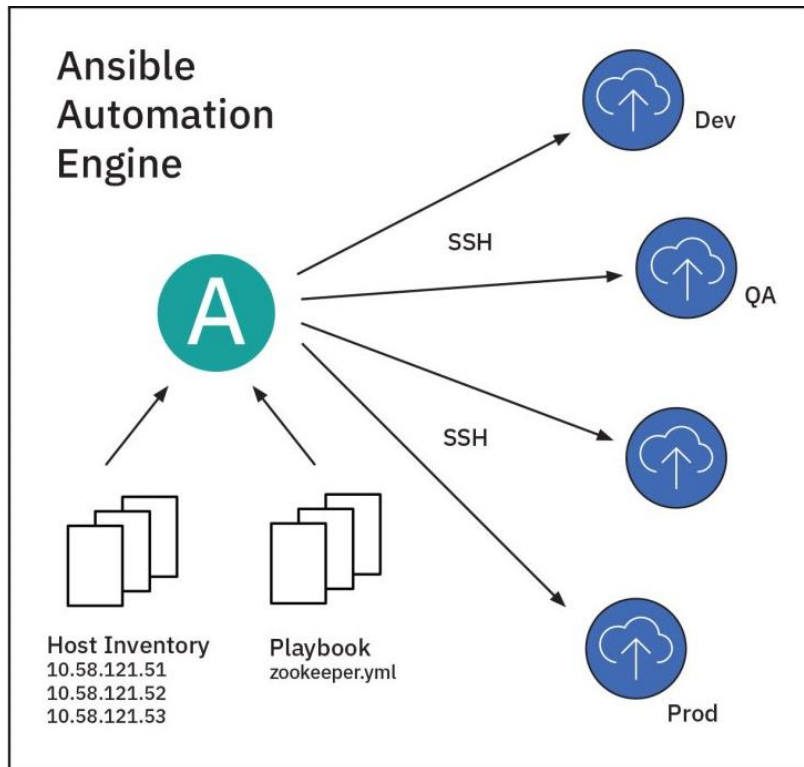


RED HAT ANSIBLE Introduction

- **Ansible** is an open-source **software provisioning, configuration management & application-deployment tool** which enabling infrastructure as code (**IaC**).
- It runs on **Unix-like** systems & can configure both **Unix-like** systems & **Windows**.
- Ansible first time released in **2013** by a Spanish Programmer (**Michael DeHaan**).
- Then it acquired by **Red Hat** Inc in **2015**, So now it is under the umbrella of **IBM**!
- Michael DeHaan before Ansible was founder of **Cobbler** & co-author of **FUNC**.
- The term "**Ansible**" was coined by a 1966 novel **Rocannon's World** & refers to fictional instantaneous communication systems.

Ansible Efficient Architecture

- **Management Node** is the controlling node which controls entire execution of the playbook. It's the node from which you are running the installation.
- **Inventory File** provides the list of hosts where the Ansible modules need to be run & the management node does an SSH connection & executes the small modules on host's machine & installs the software.
- **Playbooks** are ordered lists of tasks, so you can run those tasks in that order repeatedly.





Ansible Environment Setup



Ansible Installation Process

- ❑ Mainly, there are two types of machines when we talk about deployment:
 - **Control Machine** – Machine from where we can manage other machines.
 - **Remote Machine** – Machines that are handled/controlled by the control machine.
- ❑ There can be **multiple remote machines** that are handled by **one control machine**.
- ❑ Ansible Control Machine Requirements:
 - I. Install Python 2 (2.6 or 2.7) or Python 3 (3.5 or Higher).
 - II. Install Python 2 or 3 on Remote Machines.
 - III. Windows does not support as a Control Machine (Just by WSL).



Ansible Installation Process

- Ansible Does not add any **database** or any **daemons** for managing remote machines.
- Ansible Does not need any **software installation** on remote machines also.
 - 1) Add a “Ansible” user on each machine.
 - 2) Configure SSH private key login between machines.
 - 3) Install Ansible:

```
[root@controller ~] # yum install ansible -y
root@controller:~# apt install ansible -y
root@controller:~# ansible --version
ansible 2.9.6
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/root/.ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.8.5 (default, Jul 28 2020, 12:59:40) [GCC 9.3.0]
```

Ansible Installation Process

- In the following, we will add the **Ansible** user on all machines:

```
root@controller:~# useradd -m -s /bin/bash -c "Ansible User" ansible
root@controller:~# passwd ansible
root@remote-1:~# useradd -m -s /bin/bash -c "Ansible User" ansible
root@remote-1:~# passwd ansible
root@remote-2:~# useradd -m -s /bin/bash -c "Ansible User" ansible
root@remote-2:~# passwd ansible
```

- Then, start to send the **Controller's Public Key** to remote machines:

```
ansible@controller:~# ssh-keygen
ansible@controller:~# ssh-copy-id ansible@remote-1
ansible@controller:~# ssh-copy-id ansible@remote-2
```

Ansible Installation Process

- Finally just add **Ansible** users on all machines in the **Sudoers** group:

```
root@controller:~# usermod -aG sudo ansible
```

```
root@controller:~# vi /etc/sudoers
```

```
%ansible ALL=(ALL:ALL) NOPASSWD: ALL
```

```
root@remote-1:~# usermod -aG sudo ansible
```

```
root@remote-1:~# vi /etc/sudoers
```

```
%ansible ALL=(ALL:ALL) NOPASSWD: ALL
```

```
root@remote-2:~# usermod -aG sudo ansible
```

```
root@remote-2:~# vi /etc/sudoers
```

```
%ansible ALL=(ALL:ALL) NOPASSWD: ALL
```



Ansible Inventory



Ansible Inventory

- Ansible reads **information** about which **machines** you want to manage from **inventory**.

```
root@controller:~# cat /etc/ansible/hosts
# Ex 1: Ungrouped hosts, specify before any group headers.
blue.example.com
192.168.100.10

# Ex 2: A collection of hosts belonging to the 'webservers' group
[webservers]
www[001:006].example.com
192.168.1.110

[db-servers]
db01.intranet.mydomain.net
10.25.1.56

[all_servers]
webservers
db-servers
```

Inventory Parameters

- There are some parameters can be used in **Ansible Inventory File**:

Parameters	Description
ansible_host =192.168.1.10	IP or DNS Name of the Host.
ansible_port =2022	SSH Default port for connecting to.
ansible_connection =ssh	Ansible Connection Type [ssh/winrm/localhost].
ansible_user =ansible	SSH User, if no user specified current user will used.
ansible_ssh_pass =Aa@123	SSH Password.

Using Inventory Parameters

- It can refer to servers in Ansible, an **alias**, like below:

```
ansible@controller:~# cat /etc/ansible/hosts

# Sample Inventory parameters

web1 ansible_host=web1.example.com ansible_connection=ssh ansible_user=root

db01 ansible_host=db01.example.com ansible_connection=winrm ansible_user=admin

mail1 ansible_host=192.168.10.3 ansible_connection=ssh ansible_ssh_pass=Aa@123

ubuntu1 ansible_host=localhost ansible_connection=localhost
```

Example: Using Inventory Parameters

- Here is an example of adding some nodes to the controller machine:

```
ansible@controller:~$ vi /home/arash/inventory.txt
    ubuntu1 ansible_host=192.168.1.121 ansible_connection=ssh ansible_user=ansible
    ubuntu2 ansible_host=192.168.1.172 ansible_connection=ssh ansible_user=ansible
ansible@controller:~$ ansible --list-hosts all -i inventory.txt
hosts (2):
    ubuntu1
    ubuntu2
ansible@controller:~$ ansible ubuntu1 -m ping -i inventory.txt
ubuntu1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Add Remote Machines to “hosts” file

- Here is an example of adding some nodes to the controller machine:

```
ansible@controller:~$ vi /etc/ansible/hosts
```

```
ubuntu1 ansible_host=192.168.1.121 ansible_connection=ssh ansible_user=ansible
```

```
ubuntu2 ansible_host=192.168.1.172 ansible_connection=ssh ansible_user=ansible
```

```
ansible@controller:~$ ansible ubuntu2 -m ping
```

```
ubuntu2 | SUCCESS => {
```

```
  "ansible_facts": {
```

```
    "discovered_interpreter_python": "/usr/bin/python3"
```

```
  },
```

```
  "changed": false,
```

```
  "ping": "pong"
```

```
}
```



Ansible Ad-hoc Commands



Ansible Ad-hoc Commands

- Ad-hoc commands can **execute** some commands on one or more servers.
- This is a fast way to interact with the **desired servers**.

```
ansible [pattern] -m [module] -a "[module arguments]"
```

```
ansible@controller:~$ ansible all -m command -a "uptime"
```

```
ubuntu1 | CHANGED | rc=0 >>
```

```
20:41:39 up 22 min, 3 users, load average: 0.00, 0.00, 0.00
```

```
ubuntu2 | CHANGED | rc=0 >>
```

```
20:41:40 up 21 min, 3 users, load average: 0.00, 0.02, 0.07
```

```
ansible@controller:~$ ansible all -a "date"
```

```
ubuntu1 | CHANGED | rc=0 >>
```

```
Thu 24 Feb 2022 08:46:06 PM UTC
```

```
ubuntu2 | CHANGED | rc=0 >>
```

```
Thu 24 Feb 2022 08:46:06 PM UTC
```

Privilege Escalation

- Ad hoc tasks can be used to reboot servers, copy files, manage packages, users and more.
 - Ad hoc tasks, like **playbooks**, use a **declarative model**, calculating and executing the actions required to reach a specified **final state**.
- For running commands using **sudo privileges** can use **-b** option:

```
ansible@controller:~$ ansible all -a "whoami"
  ubuntu1 | CHANGED | rc=0 >>
  ansible
  ubuntu2 | CHANGED | rc=0 >>
  ansible

ansible@controller:~$ ansible all -b -a "whoami"
  ubuntu1 | CHANGED | rc=0 >>
  root
  ubuntu2 | CHANGED | rc=0 >>
  root
```

Some Ad-hoc Examples

- Instead of **command** module can use **shell** module also:

```
ansible@controller:~$ ansible all -m shell -b -K -a "reboot"
```

```
BECOME password:
```

```
ansible@controller:~$ ansible all -b -a "shutdown -h"
```

```
ansible@controller:~$ ansible web-srv -b -m apt -a "name=apache2 state=present"
```

```
ansible@controller:~$ ansible web-srv -b -m apt -a "name=apache2 state=latest"
```

```
ansible@controller:~$ ansible db-srv -b -m apt -a "name=apache2 state=absent"
```

```
ansible@controller:~$ ansible web-srv -b -m service -a "name=apache2 state=started"
```

Reloaded , Restarted, Running, Started or Stopped

Ansible “Command” Module

- The **command** module runs the **local binary file** you supply it.
- It is **not** run through the shell, so **no environment variable** or **piping** are not an option.

```
ansible@controller:~$ ansible ubuntu1 -m command -a 'echo "test" > ~/test.txt'
ubuntu1 | CHANGED | rc=0 >>
by command > /home/ansible/test.txt

ansible@ubuntu1:~$ cat test.txt
cat: test.txt: No such file or directory
```

Ansible “Shell” Module

- The **Shell** module will run commands via the **local shell** on the remote system.
- This is usually discouraged also in favor of the command module.
- If the command you want to execute should use **local environment variables** or you want to **pipe the output** to a file or grep (“<”, “>”, “[”, “;”, “&”), then this is the droid you are looking for.

```
ansible@controller:~$ ansible ubuntu1 -m shell -a 'echo "test" > ~/test.txt'
ubuntu1 | CHANGED | rc=0 >>
```

```
ansible@ubuntu1:~$ cat test.txt
test
```

Ansible “Raw” Module

- The **raw** module is the **most stripped down** base way to run commands on a remote system and is **NOT** recommended.
- It actually executes commands **natively** on the system and returns **standard output** back.
- This module, unlike all others, doesn't use python on the remote system.
- This means it could be used on devices like **IoT kit** or **networking devices** that don't have any other supported methods.

```
ansible@controller:~$ ansible ubuntu1 -m raw -a 'echo "test" > ~/test.txt'
```

```
ubuntu1 | CHANGED | rc=0 >>
```

```
Shared connection to 192.168.1.121 closed.
```

```
ansible@ubuntu1:~$ cat test.txt
```

```
test
```

File Management with Ansible

- Ansible Ad-hoc commands can use to doing some **file management** tasks:

- ☐ **Transferring file to many servers/machines:**

```
ansible@host:~$ ansible all -m copy -a "src=/etc/passwd dest=/tmp"
```

- ☐ **Creating a new file/directory:**

```
ansible@host:~$ ansible all -m file -a "dest=/tmp/new mode=777 owner=ansible  
group=ansible state=directory"
```

- ☐ **Deleting whole directory & files:**

```
ansible@host:~$ ansible all -m file -a "dest=/tmp/new state=absent"
```

- ☐ **Gathering Facts:**

- This module is automatically called by playbooks to gather useful variables about remote hosts. It can also be executed directly to check what variables are available to a host.

```
ansible@host:~$ ansible all -m setup
```



Ansible Playbooks



Ansible Playbooks

- Ansible Playbooks are the **way of sending commands** to remote systems through scripts.
- Ansible playbooks are used to configure complex system environments to **increase flexibility** by executing a script to one or more systems.
- They are like a **to-do list** for Ansible that contains a list of tasks.
- Each playbook is an aggregation of one or more **plays** in it.
- **Play** Defines a set of **activities (tasks)** to be run on hosts.
- The function of a play is to map a **set of instructions** defined against **a particular host**.

Create an Ansible Playbook

- Let's start by writing a **sample YAML** file:

```
- name: playbook1
  hosts: all
  become: yes

  vars:
    apache_home : /etc/apache2

  tasks:
    - name: Getting uptime
      command: uptime
```

```
- name: install apache2
  apt:
    name: apache2
    state: present

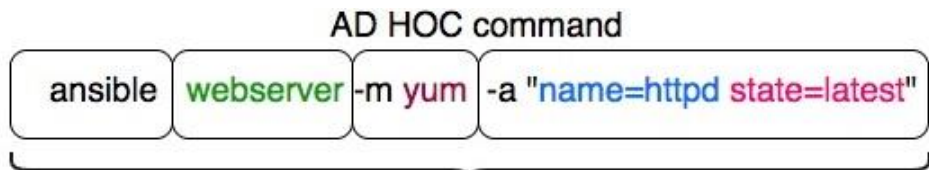
- name: start web server
  service:
    name: apache2
    state: started
```

The Different YAML Tags

- ❑ **name:** It specifies name of playbook. As in what this playbook will be doing.
- ❑ **hosts:** It specifies lists of hosts or groups which we want to run the task. This field is mandatory. It tells Ansible on which hosts to run the listed tasks.
- ❑ **vars:** Vars tag lets you define the variables which you can use in your playbook. Usage is similar to variables in any programming language.
- ❑ **tasks:** All playbooks should contain tasks or a list of tasks to be executed. Tasks are a list of actions needs to perform. A tasks field contains the name of the task. Each task internally links to a piece of code called a module. A module that should be executed, and arguments that are required for the module you want to execute.

Create an Ansible Playbook

- `ansible-playbook` (Declarative)
- `ansible-playbook <playbook name>`



Ansible Playbook

```
---
- name: playbook name
  hosts: webserver
  tasks:
    - name: name of the task
      yum:
        name: httpd
        state: latest
```

```
1 -
2   name: Play1
3   hosts: centos
4   tasks:
5     - name : Execute command 'date'
6       command: date
7
8     - name : Execute script on server
9       script: my_script.sh
10
11 -
12   name: Play2
13   hosts: centos
14   tasks:
15     - name : Install httpd service
16       yum:
17         name: httpd
18         state: present
19
20     - name : Start web server
21       service:
22         name: httpd
23         state: started
```



Ansible Playbooks Examples!



Running a Ping Test to Hosts

```
ansible@controller:~$ vi playbook-ping.yaml
```

```
  name: Ping Test Connectivity
  hosts: all
  tasks:
    - name: Ping Test
      ping:
```

```
ansible@controller:~$ ansible-playbook playbook-ping.yaml
```

```
PLAY [Ping Test Connectivity]
```

```
*****
```

```
TASK [Gathering Facts]
```

```
*****
```

```
ok: [ubuntu1]
```

```
ok: [ubuntu2]
```

```
TASK [Ping Test]
```

```
*****
```

```
ok: [ubuntu2]
```

```
ok: [ubuntu1]
```

Copy Files to Remote Hosts

```
ansible@controller:~$ vi playbook-copy.yaml
```

```
  name: Copy Files to Remote Server
```

```
  hosts: all
```

```
  tasks:
```

```
    - name: Copying Files
```

```
      copy:
```

```
        src: /etc/passwd
```

```
        dest: /tmp/
```

```
        force: yes
```

```
        backup: yes
```

```
ansible@controller:~$ ansible-playbook playbook-copy.yaml -v or -vv,-vvv,-vvvv
```

```
PLAY [Copy Files to Remote Server]
```

```
*****
```

```
TASK [Gathering Facts]
```

```
*****
```

```
ok: [ubuntu1]
```

```
ok: [ubuntu2]
```

```
TASK [Copying Files]
```

```
*****
```

```
...
```

Privilege Escalation

- For privilege escalation in playbooks, can run ansible-playbook command with **-b option**.
- Or we can add a line, **become: true** or **become: yes**.

```
ansible@controller:~$ vi playbook-apt.yaml
  name: playbook1
  hosts: ubuntu1
  become: yes
  become_user: ansible

  tasks:
    - name: install apache2
      apt:
        name: apache2
        state: present
ansible@controller:~$ ansible-playbook playbook-apt.yaml -vv
```

Ansible Playbook Handlers

```
ansible@controller:~$ vi playbook-handler.yaml
hosts: all
become: yes
tasks:
  - name: install vsftpd on ubuntu
    apt: name=vsftpd update_cache=yes state=latest
    ignore_errors: yes
    notify: start vsftpd

  - name: install vsftpd on centos
    yum: name=vsftpd state=latest
    ignore_errors: yes
    notify: start vsftpd

handlers:
  - name: start vsftpd
    service: name=vsftpd enabled=yes state=started

ansible@controller:~$ ansible-playbook playbook-handler.yaml -v
```



Ansible Modules



Ansible Modules

- Ansible will connect to nodes & push them out small programs, called **modules**.
- These programs are written to be **resource models** of the **desired state**.
- Ansible then executes these modules and removes them when finished.
- Ansible modules are categorized into various groups based on their functionality.
- There are hundreds of Ansible modules available.
- <https://github.com/ansible/ansible/tree/devel/lib/ansible/modules>
- Run **ansible-doc <module-name>** to get more information about any module.

Some Ansible Module Categories

- There are some Ansible famous modules in below table:

Categories	Module Categories							
System	User	Group	Iptables	Mount	Ping	Systemd	Service	Hostname
Commands	Command	Expect	Raw	Script	Shell			
Files	Acl	Archive	Find	Copy	Replace	Stat	File	Unarchive
Database	MySQL	MongoDB	MSSQL	PostgreSQL	ProxySQL	Vertica		
Cloud	Amazon	Azure	Google	Linode	OpenStack	VMware	Docker	Atomic
Windows	Win_copy	Win_command	Win_msi	Win_ping	Win_msq	Win_shell	Win_path	Win_service

Ansible Modules

- **System** : System modules are actions to be performed at a system level such as modifying users/groups on the system, modifying firewall, working with LVMs & mounting.
- **Command** : Command module are used to execute command/script on hosts. It can be a simple command using command module or an interactive execution using expect by responding to prompts. A script can also run on hosts using the script module.
- **Files** : Files module do some tasks like, use ACL to set an acl information on files, use archive/unarchive to compress/unpack files, use find, lineinfile & replace.
- **Database** : Database module helps in working with databases such as mongodb, mysql, mssql, postgresql, proxysql & vertica to add/remove databases or modifying them.
- **Cloud** : The Cloud section has a vast collection of modules for different cloud providers like Amazon, Azure, Google, Docker, VMware, Digital Ocean, Openstack & many more.
- **Windows** : Some of the Windows module are Win_copy to copy files, Win_command to execute a command, configuring a domain, IIS, registry & lot more.

Ansible COMMAND Module

- Command Modules **executes** a command on a remote node.
- It is a **default module** if no modules specified.

Parameter	Comments
chdir	cd into this directory before running the command.
creates	This step will not be run, when it already exists.
executable	Should be an absolute path to the executable in shell.
free_form	The command module takes a free form command to run.
removes	This step will not be run, when it doesn't exist.
warn	If warning are on in ansible.cfg, do not warn about a particular line if set to false.



Ansible Modules Examples!



Ansible Module Examples!

❑ Ansible SCRIPT Module Example:

```
ansible@controller:~$ ansible-playbook script-module.yaml
```

```
---
```

```
#Sample Ansible script-module.yaml
```

```
name: Play Scripts
```

```
hosts: centos
```

```
become: yes
```

```
tasks:
```

```
- name: Run a script on remote server
```

```
  script: /home/user1/demo-module/script.sh
```

Ansible Module Examples!

❑ Ansible SERVICE Module Example:

```
ansible@controller:~$ ansible-playbook service-module.yaml
```

```
---
```

```
#Sample Ansible service-module.yaml
```

- name: Start some Services in order
hosts: centos
become: yes
tasks:
 - name: Start the database service
service:
 - name: postgresql
 - state: started
 - enabled: yes
 - name: Start the httpd service
service: name=httpd state=started enabled=yes

Ansible Module Examples!

❑ Ansible LINEINFILE Module Example:

```
ansible@controller:~$ ansible-playbook lineinfile-module.yaml
```

```
---
```

```
#Sample Ansible lineinfile-module.yaml
```

```
name: Add DNS server
```

```
hosts: localhost
```

```
tasks:
```

```
- name: Add DNS server to resolv.conf
```

```
  lineinfile:
```

```
    path: /etc/resolv.conf
```

```
    line: 'nameserver 8.8.8.8'
```

Ansible Module Examples!

❑ Ansible MAIL Module Example:

```
ansible@controller:~$ ansible-playbook mail-module.yaml
```



mail-playbook.txt

```
name: sending mail
hosts: localhost
tasks:
  - name: sending mail to root
    mail:
      subject: 'System has been successfully configured'
      delegate_to: localhost

  - name: Sending an e-mail using Gmail SMTP servers
    mail:
      host: smtp.gmail.com
      port: 587
      username: ansible@sematec.com
      password: mysecret
      to: John Smith <john.smith@example.com>
      subject: Ansible-report
      body: 'System has been successfully provisioned.'
      delegate_to: localhost
```

Ansible Module Examples!

❑ Ansible YUM Module Example:

```
ansible@controller:~$ ansible-playbook yum-module.yaml
```

```
name: Install package(s) using yum
hosts: centos
become: yes
tasks:
  - name: Install Apache & Postgresql
    yum:
      name:
        - httpd
        - postgresql
        - postgresql-server
      state: present

  - name: Install apache >= 2.4
    yum:
      name: httpd>=2.4
      state: present
```

Ansible Module Examples!

❑ Ansible FIREWALL Module Example:

```
ansible@controller:~$ ansible-playbook firewall-module.yaml
```

```
name: Set Firewall Configurations
hosts: centos
become: yes
tasks:
  - firewallld:
      service: https
      permanent: true
      state: enabled

  - firewallld:
      port: 8080/tcp
      permanent: true
      state: disabled

  - firewallld:
      source: 192.168.100.0/24
      zone: internal
      state: enabled
```



Variables in Ansible



Variables in Ansible

- Just like any other Scripting or programming language, there are **variables** in **playbooks**.
- Variables could store different **values** for different **items**.
- Variables must **start with a letter**, only contain **letters, numbers & underscores**.
- Variables can be defined in three below scopes:
 - 1) **Global Scope:** Variables set from the Command-Line or Ansible Configuration.
 - 2) **Play Scope:** Variables set in the Play & relates structures.
 - 3) **Host Scope:** Variables set on Host Groups & Individual Hosts by inventory.

“Host Scope” Variables

- Here is a firewall configuration playbook sample which has written by variables:

```
ansible@controller:~$ vi variable-inventory.txt
centos http_port=8080 snmp_port=161-162 internal_ip_range=192.168.100.0
ansible@controller:~$ vi firewall-playbook.yaml

name: Set Firewall Configurations
hosts: centos
become: true
tasks:
  - firewallld:
      service: https
      permanent: true
      state: enabled

  - firewallld:
      port: "{{ http_port }}/tcp"
      permanent: true
      state: disabled
```

Host Scope Variables

- firewallld:
 - port: "{{ snmp_port }}/udp"
 - permanent: true
 - state: disabled
- firewallld:
 - source: "{{ internal_ip_range }}/24"
 - permanent: true
 - zone: internal
 - state: enabled

```
ansible@controller:~$ ansible-playbook -i variable-inventory.txt firewall-  
playbook.yaml
```

“Play Scope” Variables (vars)

- In play scope, variables can be defined in two formats, **vars** or **vars_file**:

```
ansible@controller:~$ vi firewall-playbook.yaml
```

```
name: Set Firewall Configurations
```

```
hosts: centos
```

```
become: true
```

```
vars:
```

```
    http_port: 8080
```

```
    snmp_port: 161-162
```

```
    internal_ip_range: 192.168.100.0
```

```
tasks:
```

```
  - firewallld:
```

```
      service: https
```

```
      permanent: true
```

```
...
```

“Play Scope” Variables (vars_file)

- In play scope, variables can be defined in two formats, **vars** or **vars_file**:

```
ansible@controller:~$ vi vars_file.yaml
```

```
http_port: 8080
```

```
snmp_port: 161-162
```

```
internal_ip_range: 192.168.100.0
```

```
ansible@controller:~$ vi firewall-playbook.yaml
```

```
name: Set Firewall Configurations
```

```
hosts: centos
```

```
become: true
```

```
vars_files:
```

```
- vars.yaml
```

```
tasks:
```

```
- firewallld:
```

```
  service: https
```

```
  permanent: true
```

```
...
```



Ansible Facts



Ansible Facts

- Ansible facts are data **gathered** about **remote** machines & **feedback** to controller.
- Ansible facts are stored in **JSON** format.
- Facts are in an **ansible_facts Variable**, which is under management of **Ansible Engine**.
- Here are some of the facts gathered:
 - Hostname, Kernel Version, Network Interfaces, IP Addresses, Version of OS, Env Variables, Number of CPUs, Available/Free Memory Space, Available Disk Space,
- Ansible Facts, use **Setup Module** for gathering facts exactly before running playbooks.

Accessing Facts Using “Ad-hoc Commands”

- **ansible all -m setup** fetches all details from remote machines to controller & dumps them directly to screen.

```
ansible@controller:~$ ansible all -m setup
```

- Specific values can be filtered from Setup Module with **-a "filter=YourFilterHere"** option.

```
ansible@controller:~$ ansible all -m setup -a "filter=*family*"
ubuntu | SUCCESS => {
  "ansible_facts": {
    "ansible_os_family": "Debian",
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false
}
```

Accessing Facts Using “Playbooks”

- In Ansible Playbooks, just needs to use actual name without using ansible keyword.
- `ansible_facts["system"]`
- The **gather_facts module** from Ansible playbook runs **setup module** by **default** at the start of each playbook to gather facts about remote hosts.
- Fetch the Ansible facts and display them using a playbook, is like this:

```
ansible@controller:~$ vi facts-playbook.yaml
- hosts: ubuntu2

tasks:
- debug:
    var: ansible_facts           Or can filter like Var: ansible_facts["cmdline"]
ansible@controller:~$ ansible-playbook facts-playbook.yaml
```

Ansible “Debug” Module

- In Ansible **playbooks**, recommended to have some **debug options**.
- Debug module used to **print messages** in the log output.

```
ansible@controller:~$ vi debug-playbook.yaml

- hosts: centos
  vars:
    - first_var: "HAhaha"

  tasks:
    - name: show results
      debug: msg="The variable first_var is set to - {{ first_var }}"

ansible@controller:~$ ansible-playbook debug-playbook.yaml
```

“Register” Variables in Ansible

- **Ansible Registers** are used for capturing **output of a task** to a **variable**.
- Registers can use in different scenarios like **conditionals**, **logging** & etc.
- The **common return values** are documented in [Ansible docs](#).

```
ansible@controller:~$ vi register-playbook.yaml
```

```
- hosts: centos
  vars:
    - var_thing: "eldorado"

  tasks:
    - name: say something
      command: echo -e "{{ var_thing }}!\n Do you believe {{ var_thing }}
                exist?!\nI like to know about {{ var_thing }}"
      register: results1

    - name: show results
      debug: msg={{ results1 }}    Filter stdout_lines: debug: msg={{ results1.stdout_lines }}
```



Conditionals in Ansible



Conditionals in Ansible

- Sometimes needed to add a **condition** to tasks.
- Here we install/remove apache package on ubuntu, but using conditions:

```
ansible@controller:~$ ansible ubuntu -b -m apt -a "name=apache2
state=absent"
ubuntu | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false
}
```

Conditions based on “Registered Variables”

- Create the name of the **registered variable** using the **register** keyword.
- A registered variable always contains the **status of the task** that created it.

```
ansible@controller:~$ vi condition1-playbook.yaml
```

```
- hosts: all
  become: yes

  tasks:
    - name: install apache2
      apt: name=apache2 state=latest
      update_cache: yes
      ignore_errors: yes
      register: variable

    - name: install httpd
      yum: name=httpd state=latest
      failed_when: "'FAILED' in variable"
```

Conditions based on “ansible_facts”

- Not all facts exist for all hosts. For example, ‘**lsb_major_release**’ fact only exists when **lsb_release** package is installed on target host.
- Remove web server package from each server based on its OS family:

```
ansible@controller:~$ vi condition2-playbook.yaml
```

```
- hosts: ubuntu,centos,suse
  become: yes

  tasks:
    - name: Remove Apache on Ubuntu Server
      apt: name=apache2 state=absent
      when: ansible_os_family == "Debian"

    - name: Remove Apache on CentOS Server
      yum: name=httpd state=absent
      when: ansible_os_family == "RedHat"
```



Loops in Ansible



Loops in Ansible

- Sometimes needed to **repeat the execution** of a same task using **different values**.
- For example, installing multiple packages , or

```
ansible@controller:~$ vi noloop-playbook.yaml
```

```
- hosts: ubuntu  
  become: yes
```

```
tasks:
```

- ```
- yum: name=vim state=present
- yum: name=nano state=present
- yum: name=apache2 state=present
```

- There are different kinds of loops. Take a look at standard ones.

# Loops in Ansible with “with\_items”

- Sometimes needed to **repeat the execution** of a same task using **different values**.
- For example, installing multiple packages , or ... .

```
ansible@controller:~$ vi loop1-playbook.yaml
```

```
- hosts: ubuntu
 become: yes
```

```
tasks:
```

```
- name: install packages
 apt: name={{ item }} update_cache=yes state=latest
 with_items:
 - vim
 - nano
 - apache2
```

# Loops in Ansible with “with\_items”

- From Version Ansible 2.11, there will be no with\_items module!!

```
ansible@controller:~$ vi loop1-playbook.yaml

- hosts: ubuntu
 become: yes

 tasks:
 - name: install packages
 apt:
 name=['vim', 'nano', 'apache2', 'curl', 'figlet']
 update_cache=yes
 state=latest
```

# Loops in Ansible with “with\_files”

- Another loop, using different files:

```
ansible@controller:~$ vi loop2-playbook.yaml
```

```
- hosts: ubuntu
 become: yes
```

```
tasks:
```

```
- name: show file(s) contents
 debug: msg={{ item }}
 with_file:
 - myfile1.txt
 - myfile2.txt
```

```
ansible@controller:~$ cat myfile1.txt
```

```
This is myfile1.txt, first line :-)
```

```
ansible@controller:~$ cat myfile2.txt
```

```
This is myfile2.txt, first line :-0
```

# Loops in Ansible with “with\_sequence”

- Another loop, using different sequence orders:

```
ansible@controller:~$ vi loop3-playbook.yaml
```

```
- hosts: ubuntu
```

```
 become: yes
```

```
 tasks:
```

```
 - name: show file(s) contents
```

```
 debug: msg={{ item }}
```

```
 with_sequence: start=1 end=5
```



# Ansible Vault



# What is Ansible Vault?

- Some of Ansible files contain **sensitive & important data** like usernames & passwords.
- **Ansible Vault** prevents this data from being exposed.
- It keeps passwords & other sensitive data in an **encrypted file**.
- Ansible Vault performs **various operations**, like:
  - Create an encrypted file
  - Decrypt a file
  - Encrypt a file
  - View an encrypted file without breaking encryption
  - Edit an encrypted file
  - Generate or reset the encrypted key

# Creating an Encrypted File

- Use **ansible-vault create** command to create the encrypted file:

```
ansible@controller:~$ ansible-vault create apache-playbook.yaml
```

```
New Vault password:
```

```
Confirm New Vault password:
```

```
ansible@controller:~$ vi apache-playbook.yaml
```

```
hosts: localhost
```

```
become: yes
```

```
vars:
```

```
- ansible_sudo_pass: Aa@12345
```

```
tasks:
```

```
- name: install apache2
```

```
 apt: name=apache2 state=latest
```

# Decrypting a File

- The **ansible-vault decrypt** command is used to decrypt an encrypted file:

```
ansible@controller:~$ ansible-vault decrypt apache-playbook.yaml
```

```
Vault password:
```

```
Decryption successful
```

```
ansible@controller:~$ cat apache-playbook.yaml
```

```
hosts: localhost
```

```
become: yes
```

```
vars:
```

```
- ansible_sudo_pass: Aa@12345
```

```
tasks:
```

```
- name: install apache2
```

```
 apt: name=apache2 state=latest
```

# Encrypting a File

- The **ansible-vault encrypt** command is used to encrypt an existing playbook file:

```
ansible@controller:~$ ansible-vault encrypt apache-playbook.yaml
New Vault password:
Confirm New Vault password:
Encryption successful
ansible@controller:~$ cat apache-playbook.yaml
```

# Decrypt a Running Playbook (--ask-vault-pass)

- During running an encrypted ansible playbook, you will get error like below:

```
ansible@controller:~$ ansible-playbook apache-playbook.yaml
ERROR! Attempting to decrypt but no vault secrets found
```

- So during running an ansible-playbook, needs to use **--ask-vault-pass** option:

```
ansible@controller:~$ ansible-playbook apache-playbook.yaml --ask-vault-pass
Vault password:

PLAY [localhost]

TASK [Gathering Facts]

ok: [localhost]
...
```

# Decrypt a Running Playbook (--vault-password-file)

- Another way is using a file which contains password with below syntax:

```
ansible@controller:~$ vi password.txt
Aa@12345
ansible@controller:~$ ansible-playbook apache-playbook.yaml --vault-password-
file password.txt

PLAY [localhost]

TASK [Gathering Facts]

ok: [localhost]

TASK [install httpd]

ok: [localhost]

...
PLAY RECAP

localhost: ok=2 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

# Decrypt a Running Playbook (--vault-id)

- Previous commands has been deprecated, new option is **--vault-id**:

```
ansible@controller:~$ ansible-playbook apache.yaml --vault-id @prompt
```

```
Vault password (default):
```

```
PLAY [localhost]
```

```

```

```
TASK [Gathering Facts]
```

```

```

```
ok: [localhost]
```

```
...
```

```
ansible@controller:~$ ansible-playbook apache.yaml --vault-id @password.txt
```

```
PLAY [localhost]
```

```

```

```
TASK [Gathering Facts]
```

```
...
```

# Encrypt the Value of the Variable

- With Ansible Vault also can **encrypt variables** only.
- Sensitive variables such as passwords and API keys can encrypt by **encrypt\_string**:

```
ansible@controller:~$ ansible-vault encrypt_string 'Aa@12345' --name
 'ansible_sudo_pass'
```

```
New Vault password:
```

```
Confirm New Vault password:
```

```
ansible_sudo_pass: !vault |
```

```
 $ANSIBLE_VAULT;1.1;AES256
```

```
 3266663333393932663233623766326238346566323266663337326339393962343938666
```

```
 36433363638373263343738613466633765383836353739383733650a6436616231316338
```

```
 6139626261386133663035363334653866623165333337623566376165663739313332333
```

```
 0333139393337393437306135653863653962350a63316139626235346235363666356665
```

```
Encryption successful
```

# Encrypt the Value of the Variable

- Copy the entire encrypted code from **!vault |** and copy it to playbook file.

```
ansible@controller:~$ vi apache-playbook.yaml
```

```
hosts: localhost
become: yes
```

```
vars:
```

```
- ansible_sudo_pass: !vault |
 $ANSIBLE_VAULT;1.1;AES256
 3266663333393932663233623766326238346566323266663337326339393962343938666
 36433363638373263343738613466633765383836353739383733650a6436616231316338
 6139626261386133663035363334653866623165333337623566376165663739313332333
```

```
tasks:
```

```
- name: install apache2
 apt: name=apache2 state=latest

- name: print a secure variable
 debug:
 var: ansible_sudo_pass
```

# View an Encrypted File

- To look at on an encrypted file, use **ansible-vault view** command:

```
ansible@controller:~$ ansible-vault view apache-playbook.yaml
Vault password:

#sample playbook to test vault secret-playbook.yaml
-
 hosts: localhost
 become: yes
 vars:
 - ansible_sudo_pass: Aa@12345

 tasks:
 - name: install apache2
 apt: name=apache2 state=latest
```

# Editing an Encrypted File / Resetting Pass

- For editing an encrypted file, use **ansible-vault edit** command:

```
ansible@controller:~$ ansible-vault edit apache-playbook.yaml
Vault password:
```

- For changing vault password of an encrypted file, use **ansible-vault rekey** command:

```
ansible@controller:~$ ansible-vault rekey apache-playbook.yaml
Vault password:
New Vault password:
Confirm New Vault password:
Rekey successful
```



# Ansible Template



# What is Ansible Template?

- Sometimes needed to transfer configuration files to hosts.
- We can use **copy module** in a **playbook** to copy files to the server.
- By **Ansible Templates** can define text files with **variables instead of static values**.
- **Ansible template** is a text file built with **Jinja2 templating language** with **.j2 extension**.
- A **Jinja2 template** looks exactly like the text file you'd like to get onto a remote host. The only difference is that **instead of static values**, the file contains **variables**.

```
ansible@controller:~$ ansible all -b -m apt -a "name=apache2 state=present"
```

# Creating & Using an Ansible Template

- Next create a template for web server default page (**index.html.j2**):

```
ansible@controller:~$ vi index.html.j2
<html>
<center>
<h1> This machine's hostname is {{ ansible_hostname }}/h1>
<h2> os family is {{ ansible_os_family }}/h2>
<small>file version is {{ file_version }}/small>
{# This is comment, it will not appear in final output #}
</center>
</html>
```

# Creating & Using an Ansible Template

- Then, create the related playbook for the template with **Template Module**:

```
ansible@controller:~$ vi template-playbook.yaml
```

```
- hosts: all
 become: yes
 vars:
 file_version: 1.0

 tasks:
 - name: install default web page
 template:
 src: index.html.j2
 dest: /var/www/html/index.html
 mode: 0777
```

# Creating & Using an Ansible Template

- Finally, run the playbook which created in the previous slide:

```
ansible@controller:~$ ansible-playbook template-playbook.yaml
PLAY [all]

TASK [Gathering Facts]

ok: [ubuntu1]
ok: [ubuntu2]

TASK [install default web page]

changed: [ubuntu1]
changed: [ubuntu2]

PLAY RECAP

ubuntu1: ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
ubuntu2: ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

# Include Statement

- To **splitting** a large playbook into smaller, **Include Statements** will use.
- Also tasks can move to a separate file and use include statement to include tasks from:

```
ansible@controller:~$ vi include-playbook.yaml
```

```

```

```
- include: pre-installation-playbook.yaml
```

```
- hosts: all
```

```
 become: yes
```

```
 tasks:
```

```
 - include: installation-playbook.yaml
```

```
 - include: post-installation-playbook.yaml
```



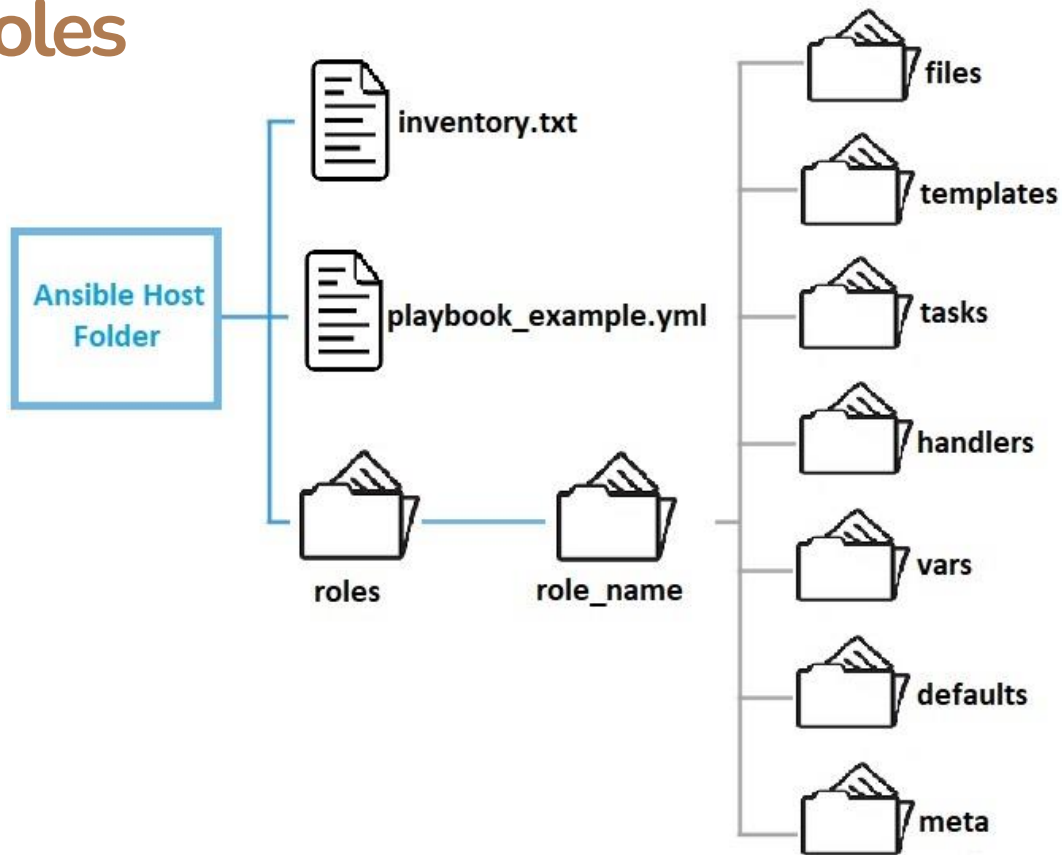
# Ansible Roles



# Ansible Roles

- In programming languages, we try to modularize our code into packages, modules, classes and functions. That way our code becomes **more organized & readable** for others.
  - When there are many inventory & playbook files, use **Ansible Roles** for organizing them.
  - **Ansible Roles** give us a **folder structure** to storing Ansible files.
  - Usually file extension on roles is **.yaml** instead of **.yml**.
- 1) Roles **break** a complicated playbook into multiple reusable **components**.
  - 2) Each **component** offers a small **function** can be used **independently** within playbook.
  - 3) So instead of create a complex play, can create **many roles & drop them** into playbooks.

# Ansible Roles



# Ansible Roles

- Lets take a look an example:

```
ansible@controller:~$ tree -F
```

```
.
├── roles/
│ ├── apache/
│ │ ├── handlers/
│ │ │ └── main-handlers.yml
│ │ └── tasks/
│ │ └── main-tasks.yml
└── site-playbook.yml
```

```
4 directories, 3 files
```

# Ansible Roles

## 1) Main-tasks.yml:

```
ansible@controller:~$ cat roles/apache/tasks/main-tasks.yml
```

- name: install on debian
  - apt: name=apache2 state=latest update\_cache=yes
  - when: ansible\_os\_family == "Debian"
  - notify: start debian service
  
- name: install on centos
  - yum: name=httpd state=latest update\_cache=yes
  - when: ansible\_os\_family == "RedHat"
  - notify: start centos service

# Ansible Roles

## 2) Main-handlers.yml:

```
ansible@controller:~$ cat roles/apache/handlers/main-handlers.yml
- name: start debian service
 service: name=apache2 enabled=yes state=started

- name: start centos service
 service: name=httpd enabled=yes state=started
```

## 3) Site-playbook.yml:

```
ansible@controller:~$ cat site-playbook.yml
- hosts: all
 become: yes
 roles:
 - apache
```

# Ansible Roles

- Lets run the playbook:

```
ansible@controller:~$ ansible-playbook site-playbook.yml
```

```
PLAY [all]
```

```

```

```
TASK [Gathering Facts]
```

```

```

```
ok: [centos]
```

```
ok: [ubuntu]
```

```
PLAY RECAP
```

```

```

```
centos: ok=1 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

```
ubuntu: ok=1 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

# Ansible Roles

- When we want to create a role, we can either create required file structure manually or we can use **ansible-galaxy** command:

```
ansible@controller:~$ ansible-galaxy init new_role
```

```
- Role new_role was created successfully
```

```
.
├── new_role
│ ├── defaults
│ │ └── main.yml
│ ├── files
│ ├── handlers
│ │ └── main.yml
│ ├── meta
│ │ └── main.yml
│ ├── README.md
│ ├── tasks
│ │ └── main.yml
│ └── .
└── .
└── .
```

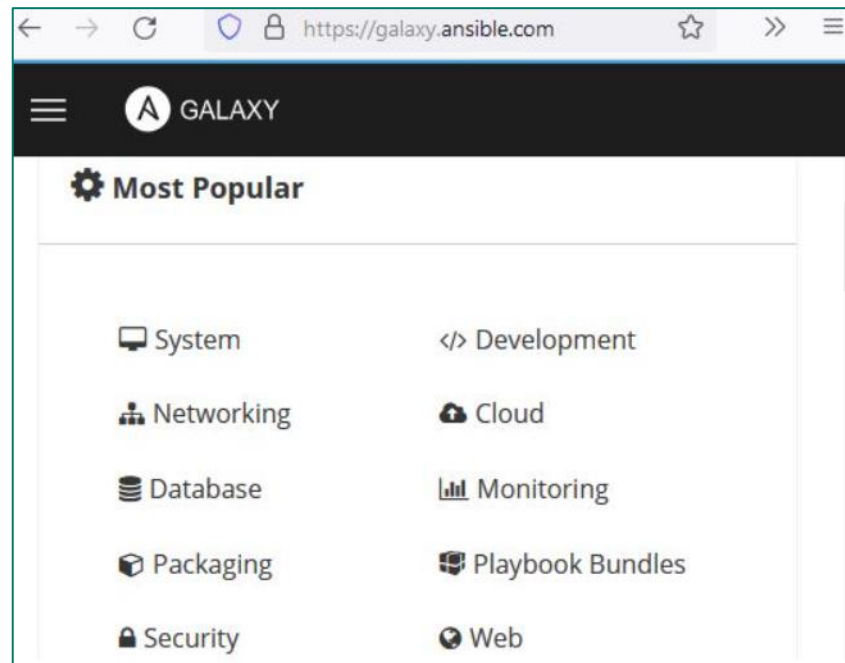


# Ansible Galaxy



# What is Ansible Galaxy?

- **Ansible Galaxy** is a galaxy website where users can share roles & to a command-line tool for **installing, creating, & managing** roles.
- It gives greater visibility to one of Ansible's features, like **application installation** or **reusable roles** for server configuration.
  - To work with Ansible Galaxy, needs to use **ansible-galaxy** command & its templates.
  - Roles must be **downloaded** before they used in the **playbooks**.
  - They are placed into **default directory** that is **/etc/ansible/roles**, also based on version, might placed in current user home.



# Search on Anisble Galaxy

- **ansible-galaxy search** searches for **roles** on the Ansible Galaxy server.

```
ansible@controller:~$ ansible-galaxy search apache
```

```
- Found 3237 roles matching your search. Showing first 1000.
```

Name	Description
----	-----
0outsider.ansible_zabbix_agent	Installing and maintaining zabbix-agent
0x0i.elasticsearch	Elasticsearch, a real-time distributed search
0x0i.kafka	Kafka, a distributed and fault tolerant ...
0x0i.systemd	Systemd, a system and service manager for Linux os
1it.docker-run	Ansible role to build and run docker containers ...
1it.riak	Installs and configures Riak KV and TS, a ...
1nfinitem.php	PHP installation role.
4linuxdevops.web-server	Instalacao e Configuracao do servidor Apache
5KYDEV0P5.common	Common Utilities and package installation for Linux
5KYDEV0P5.skydevops-java	Installs and configures Oracle Java 1.8.x
5KYDEV0P5.skydevops-maven	Install and configure Apache Mvaen 3.x.x
...	

# Download from Anisble Galaxy

- Use the **ansible-galaxy install** command to **download roles** from the Galaxy server:

```
ansible@controller:~$ ansible-galaxy install geerlingguy.apache
 geerlingguy.mysql

- downloading role 'apache', owned by geerlingguy
- downloading role from https://github.com/geerlingguy/ansible-role-
 apache/archive/3.1.4.tar.gz
- extracting geerlingguy.apache to /home/user1/.ansible/roles/geerlingguy.apache
- geerlingguy.apache (3.1.4) was installed successfully
- downloading role 'mysql', owned by geerlingguy
- downloading role from https://github.com/geerlingguy/ansible-role-
 mysql/archive/3.3.2.tar.gz
- extracting geerlingguy.mysql to /home/user1/.ansible/roles/geerlingguy.mysql
- geerlingguy.mysql (3.3.2) was installed successfully
```

# Download from Anisble Galaxy

- Lets take look at what it has downloaded:

```
ansible@controller:~$ ansible-galaxy install geerlingguy.apache
 geerlingguy.mysql
ansible@controller:demo-galaxy$
ansible@controller:demo-galaxy$ ls /home/user1/.ansible/roles/
 geerlingguy.apache geerlingguy.mysql
ansible@controller:demo-galaxy$
ansible@controller:demo-galaxy$ cd /home/user1/.ansible/roles/
ansible@controller:roles$
ansible@controller:roles$ cd geerlingguy.apache/
ansible@controller:geerlingguy.apache$ ls
 defaults handlers LICENSE meta molecule README.md tasks templates vars
ansible@controller:demo-galaxy$
ansible@controller:geerlingguy.apache$ cd ~/demo-galaxy/
ansible@controller:demo-galaxy$
```

# Download from Anisble Galaxy

- For using downloaded role, we have to **create a simple playbook**:

```
ansible@controller:demo-galaxy$ vi galaxy-playbook.yaml
```

```

```

```
#Sample playbook for galaxy and roles galaxy-playbook.yaml
```

```
- hosts: all
 become: yes
 roles:
```

```
 - geerlingguy.apache
 - geerlingguy.mysql
```

```
ansible@controller:demo-galaxy$ ansible-playbook galaxy-playbook.yaml
```

# Upload to Galaxy

- To upload a role to **Ansible Galaxy**, first you have to upload it to your **github project** and next log in to your **ansible galaxy account** and **pull that**, That's all!

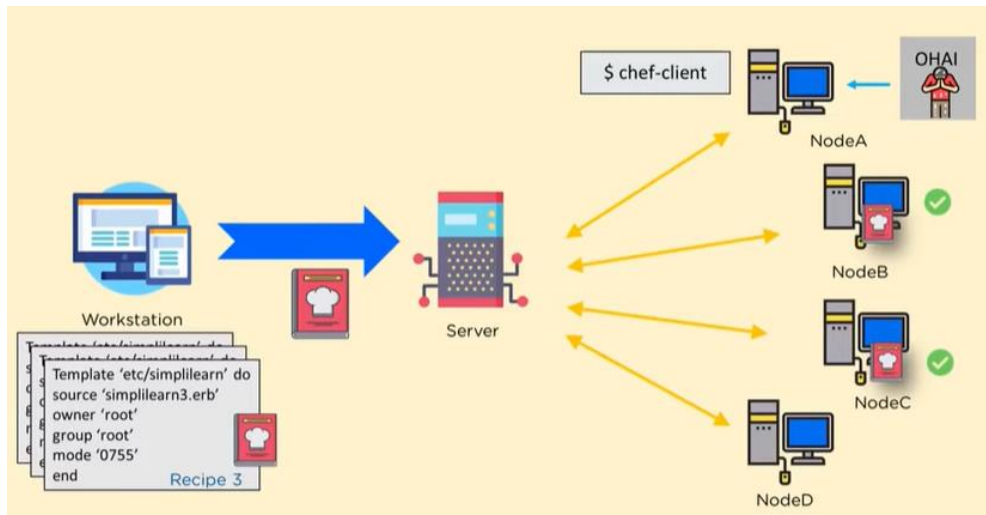


# Chef vs. Puppet vs. Ansible vs. Saltstack vs. Terraform



# When Chef is Preferred?

- Chef ensures configurations are applied **consistently** in every environment, at any **scale**, with infrastructure **automation**.
- Chef is an open source tool developed by **Opscode** written in **Ruby & Erlang**.
- Chef is best suited for organizations that have a **heterogenous infrastructure**.

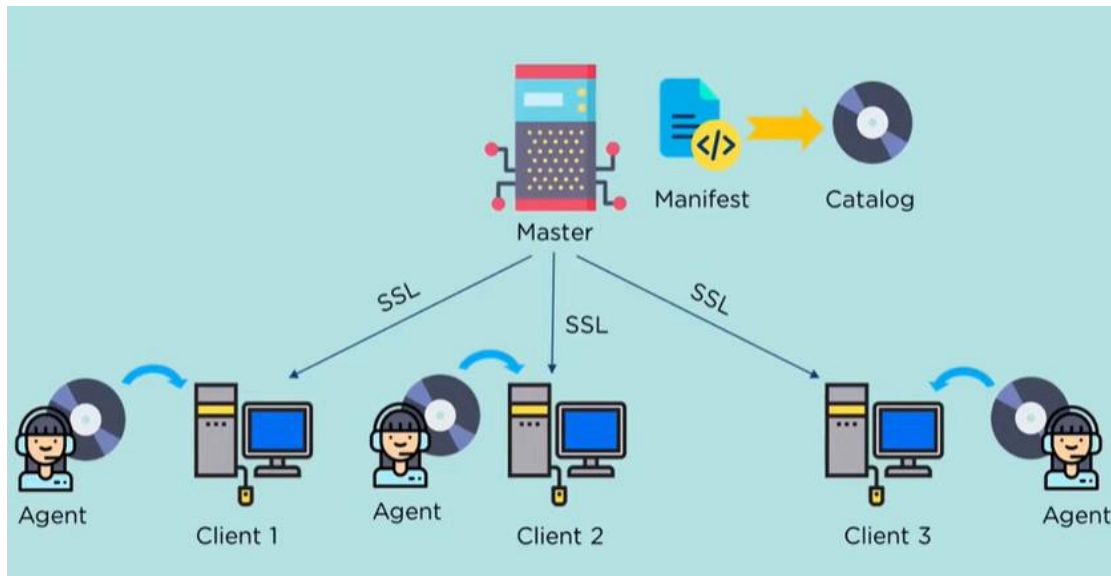


# Chef Goods & Bads!

Goods	Bads
A large collection of recipes are available.	A considerable amount of learning time needed if one is not comfortable with Ruby.
Integrates well with Git which provides a strong version controlling.	The main server doesn't have much control.

# When Puppet is Preferred?

- Puppet is a configuration management tool that supports automatic deployment.
- Puppet is built in **Ruby** & uses **DSL** for writing **manifests**.
- Puppet also works well with heterogenous environments where the focus is on scalability.

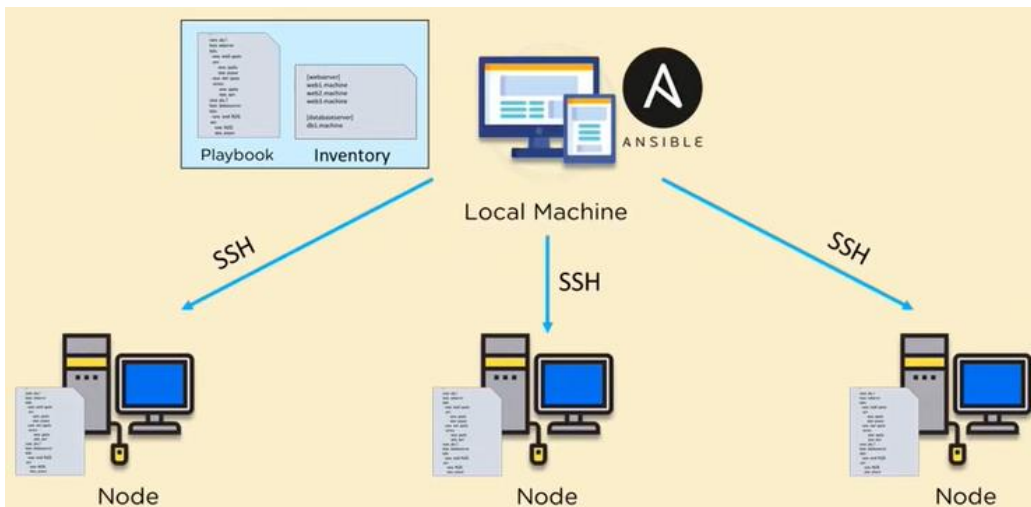


# Puppet Goods & Bads!

Goods	Bads
Strong community supported by <b>Puppet Labs</b> .	For performing advanced tasks, a good knowledge of Ruby is required.
Well developed reporting mechanism.	The main server doesn't have much control.

# When Ansible is Preferred?

- Ansible is an IT Automation tool that automates configuration management, cloud provisioning, deployment and orchestration.
- The core of Ansible, playbooks are written in **YAML** format.
- Ansible works well with environments where the focus is on getting servers up & running quickly.



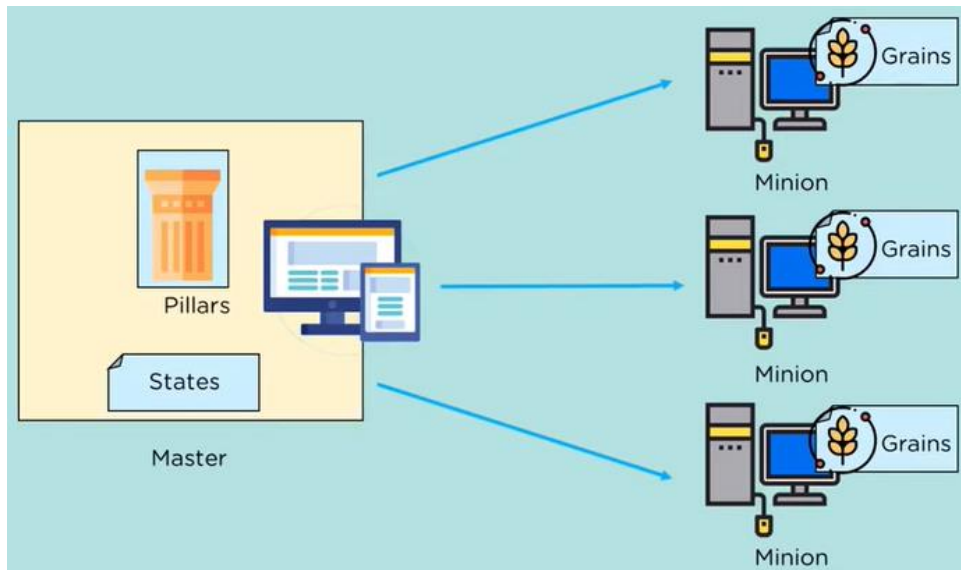
ANSIBLE

# Ansible Goods & Bads!

Goods	Bads
Agent need not be installed on the systems that require configuration.	Performance speed is often less than other tools.
YAML is extremely easy to understand and learn.	YAML is not as powerful as most other languages.

# When Saltstack is Preferred?

- Saltstack is a CLI tool that automates configuration management & remote execution.
- Saltstack is **Python** based while the **instructions** are written in **YAML** or it's **DSL**.
- Saltstack is perfect for an environment with scalability & resilience as its priority.



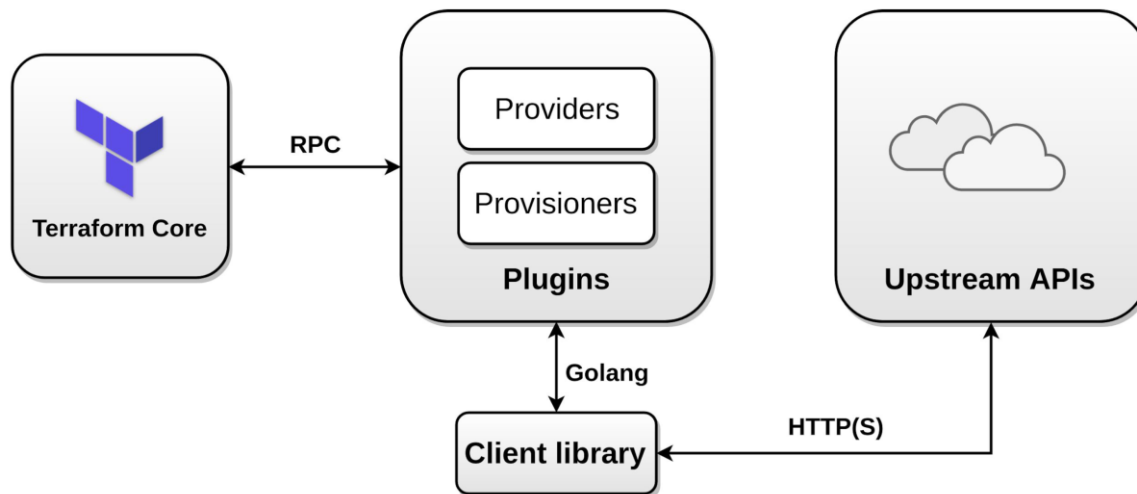
# SALTSTACK

# Saltstack Goods & Bads!

Goods	Bads
Extremely simple usage once it's set up.	Setup phase is slightly tougher.
A good reporting mechanism that allows one to easily view all operations.	Relatively new web UI which is much less developed as compared to other tools.

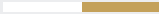
# When Terraform is Preferred?

- Terraform can manage infrastructure on multiple cloud platforms.
- **Human-readable configuration language** helps you write infrastructure code quickly.
- Terraform's state allows you to **track resource changes** throughout your deployments.
- Configurations can commit to version control to safely collaborate on infrastructure.



# Comparing All Together!

	Chef	Puppet	Ansible	Saltstack	Terraform
Architecture	Server/Client	Server/Client	Server Only	Server/Client	Server/Client
Ease of Setup	Moderate	Moderate	Very Easy	Moderate	Very Easy
Language	Specify how to do a task	Specify only what to do	Specify how to do a task	Specify only what to do	Specify how to do a task
Scalability	Scalable	Scalable	Scalable	Scalable	Scalable
Management	Tough as it requires to learn Ruby DSL	Tough as it requires to learn Puppet DSL	Very Easy by YAML	Agnostic & Simple	JSON (YAML)
Interoperability	High	High	High	High	High
Cloud Availability	Amazon	Amazon, Azure	Amazon	None	Amazon
Communication	Knife Tool	SSL	SSH	SSH	SSH



Thank you.

